

CONTENTS

Arquitectura de Von Neumann y Organización de la CPU	
Módulo 5 — la guía de tolerancia cero para no perder el parcial una tercera vez	
0. Por qué existe este problema	
1. Los tres bloques de una computadora (Von Neumann)	
1.1 Arquitectura de Harvard (variante)	
2. Dentro de la CPU: ALU, Unidad de Control, Banco de Registros	
2.1 Los tres niveles de visibilidad de los registros	
3. CISC vs RISC — la tabla que hay que saber de memoria	
4. Formato de instrucción y cálculo de bits	
4.1 Procedimiento para calcular cuántos bits hacen falta	
4.2 Ejemplo integrador: armando un formato de instrucción completo	
5. Modos de direccionamiento	
5.1 Inmediato	
5.2 Directo a registro	
5.3 Directo a memoria	
5.4 Indirecto a registro	
5.5 Indirecto a memoria	
5.6 Tabla resumen de los 5 modos	
6. Ciclo de instrucción	

6.1 Las 5 etapas	
6.2 El “Fetch” no siempre dura lo mismo	
7. Lógica cableada vs. lógica microprogramada	
7.1 Lógica cableada (hardwired)	
7.2 Lógica microprogramada	
7.3 El micro-ensamblador	
8. Ejercicios de repaso rápido (con respuesta)	
9. Resumen ultra-rápido (2 minutos)	

Arquitectura de Von Neumann y Organización de la CPU

MÓDULO 5 — LA GUÍA DE TOLERANCIA CERO PARA NO PERDER EL PARCIAL UNA TERCERA VEZ

Arquitectura de Computadoras — FING UdelaR

CPU · Memoria · E/S · Buses · ALU · Unidad de Control · Banco de Registros · CISC vs RISC · Formato de instrucción · Modos de direccionamiento · Ciclo de instrucción · Lógica cableada vs microprogramada

Cómo usar esta guía: cada tema tiene definición corta, procedimiento paso a paso, ≥ 2 ejemplos resueltos y verificación explícita, tabla resumen y un recuadro rojo con el error típico de examen. Toda cuenta de bits se verifica con $\log_2$ y una comprobación numérica — no hay ninguna cifra “porque sí” en esta guía.

0. Por qué existe este problema

Ya sabés (Módulo 1) que todo dato se reduce a una tira de bits. Ahora la pregunta es distinta: **¿cómo hace la computadora para saber qué hacer con esos bits?**

Von Neumann resolvió esto con una idea que hoy nos parece obvia pero que en 1945 fue revolucionaria: **el programa es un dato más**. Las instrucciones que la máquina va a ejecutar están guardadas en la misma memoria que los datos que va a procesar, como una secuencia ordenada de códigos binarios. Esto significa que hace falta:

1. Un lugar físico donde estén programa y datos: la **Memoria**.
2. Un circuito que sepa leer esas instrucciones, una por una, en orden, y ejecutarlas: la **CPU**.
3. Un mecanismo para que la máquina hable con el mundo exterior: la **E/S**.
4. Un conjunto de “camino” (cables agrupados) que conecten estas tres piezas: los **buses**.

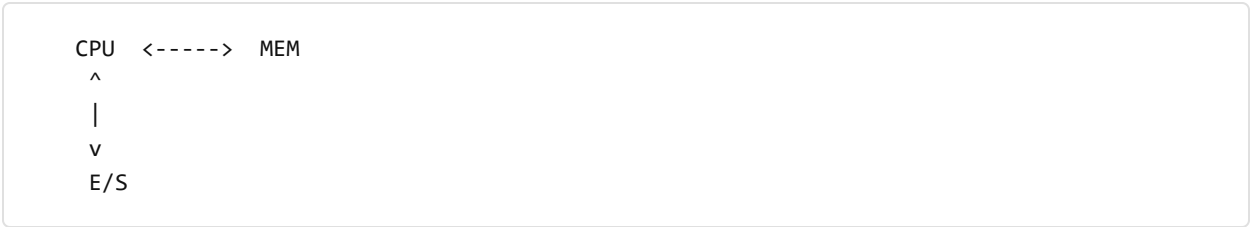
Idea que hay que tener siempre presente: la CPU no “entiende” instrucciones como C++ o Python. Entiende códigos binarios fijos, definidos de antemano por el diseñador del hardware. Todo lo que hace esta guía es explicar cómo se diseñan esos códigos y qué circuito los ejecuta.

Este módulo cubre exactamente esas dos preguntas: **cómo se organiza la CPU por dentro** (para ejecutar instrucciones) y **cómo se diseña el formato de esas instrucciones** (para que la CPU las pueda decodificar). Es la base de todo lo que viene después en el curso (pipeline, superescalares), así que un error de base acá se arrastra a todo el resto.

1. Los tres bloques de una computadora (Von Neumann)

Definición: una Arquitectura de Von Neumann tiene tres bloques constructivos conectados por un bus:

Bloque	Función
CPU	Ejecuta los programas (instrucciones)
Memoria	Almacena el programa y los datos
E/S	Comunica el sistema con el mundo exterior



El bus que conecta todo esto se subdivide en tres:

- **Bus de Direcciones:** transporta qué posición de memoria (o de E/S) se quiere acceder.
- **Bus de Datos:** transporta el dato que se lee o escribe.
- **Bus de Control:** transporta señales de sincronismo y comando (ej.: `Memory_Read`, `Memory_Write`, `Clock`, `Reset`).

Error frecuente: confundir “bus de direcciones” con “dirección de memoria”. El bus de direcciones es el conjunto de cables físicos; una dirección concreta (ej.: 0x00FF) es el valor que se pone en ese bus en un instante dado. Si te preguntan “¿cuántas líneas tiene el bus de direcciones para direccionar 1024 posiciones?”, la respuesta es $\log_2(1024) = 10$ líneas — es una cuenta de bits, no una definición.

1.1 ARQUITECTURA DE HARVARD (VARIANTE)

La arquitectura de Harvard separa la memoria en dos: una para instrucciones y otra para datos, cada una con su propio bus. **No cambia la esencia de Von Neumann**: sigue habiendo un programa (secuencia lógica de instrucciones) ejecutado por una CPU. Los procesadores modernos usan una “Harvard modificada”: caminos separados para acceder a instrucciones y datos simultáneamente, pero compartiendo la memoria física subyacente (típicamente a través de cachés L1 separadas de instrucciones y de datos).

2. Dentro de la CPU: ALU, Unidad de Control, Banco de Registros

Definición: la CPU se organiza en tres subsistemas:

- **ALU (Arithmetic Logic Unit):** circuito combinatorio que ejecuta las operaciones aritméticas y lógicas (suma/resta en complemento a 2, AND/OR/XOR/NOT bit a bit, desplazamientos).
- **Unidad de Control (UC):** circuito secuencial que ejecuta el **ciclo de instrucción** — busca la próxima instrucción, la decodifica, lee operandos, ordena a la ALU ejecutar y guarda el resultado.
- **Banco de Registros:** un conjunto pequeño de posiciones de memoria de acceso muy rápido, físicamente dentro de la CPU (no en la memoria principal).

¿Por qué existe el Banco de Registros si ya hay memoria principal? Por dos razones tecnológicas, no por capricho de diseño: (1) la velocidad de transferencia está limitada por la distancia física — los registros están pegados a la ALU; (2) la memoria rápida es más cara, así que se usa poca (los registros) y se ubica donde más se necesita.

2.1 LOS TRES NIVELES DE VISIBILIDAD DE LOS REGISTROS

Esta es una distinción que suele preguntarse directamente en examen y que se contesta mal por apurarse:

Categoría	Qué es	Ejemplos	Quién lo maneja
Totalmente visibles	Registros de uso general (o “con personalidad”) que contienen operandos o direcciones y aparecen explícitamente en las instrucciones	AX, BX, R1, R2, ...	El programador de bajo nivel, directamente
Parcialmente visibles	Tienen función especial y participan indirectamente — no se nombran en cada instrucción, pero cambian como efecto de ellas	IP/PC (próxima instrucción), SP (tope de pila), PS/FLAGS (bits de estado: Z, N, C, V)	El programador, en forma indirecta (ej.: un <code>CALL</code> modifica el SP sin que el programador lo escriba explícitamente)
Internos (no visibles)	Uso exclusivo de la Unidad de Control para ejecutar instrucciones: constantes, estado de la UC, la instrucción en curso, resultados intermedios de cálculo de direcciones	Registro IR (Instruction Register), registros auxiliares A, B, ... de la UC	Nadie fuera de la UC — el programador no puede acceder ni influir en ellos

Ejemplo: en la instrucción `ADD R1, R2, R3`, los registros R1, R2 y R3 son **totalmente visibles** (aparecen explícitos). Al ejecutarse esta instrucción, el registro **IP** (parcialmente visible) se incrementa automáticamente para apuntar a la siguiente instrucción, sin que el programador lo haya escrito en el código. Mientras tanto, la Unidad de Control puede usar un registro interno (invisible) para guardar temporalmente el valor leído de memoria antes de decodificarlo — ese registro jamás aparece en ningún manual de instrucciones para el programador.

Error frecuente: decir que el IP es “totalmente visible” porque el programador “sabe” que existe. La clave no es si el programador conoce su existencia, sino si el registro **aparece como operando explícito** en el formato de instrucción. El IP cambia como consecuencia de ejecutar instrucciones (saltos, incrementos automáticos), pero no se nombra directamente en una instrucción aritmética común — por eso es parcialmente visible, no totalmente.

3. CISC vs RISC — la tabla que hay que saber de memoria

Definición: son dos filosofías de diseño del set de instrucciones de una CPU. Surgen porque en 1980 estudios de Berkeley y Stanford mostraron que “menos instrucciones, bien optimizadas” podía rendir mejor que “muchas instrucciones complejas”.

Aspecto	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Cantidad de instrucciones	Muchas (cientos)	Pocas (decenas)
Complejidad de cada instrucción	Alta: una instrucción puede hacer varias cosas (ej.: comparar strings, buscar en un array)	Baja: cada instrucción hace una operación simple y atómica
Formato de instrucción	Largo variable	Largo fijo
Acceso a memoria desde instrucciones aritméticas	Sí — se puede sumar directamente un operando en memoria (ej.: <code>ADD [dir], AX</code>)	No — arquitectura load-store : solo <code>LOAD</code> / <code>STORE</code> tocan memoria; las aritméticas trabajan siempre entre registros
Cantidad de registros de uso general	Pocos, muchas veces “con personalidad” (AX=acumulador, CX=contador, ...)	Muchos, de propósito general, intercambiables
Modos de direccionamiento	Muchos y complejos	Pocos y simples
Unidad de Control típica	Microprogramada	Cableada
Ejemplo real	Intel x86	MIPS, SPARC, ARM

Truco para no confundirte en el parcial: la pregunta más común es “¿esta arquitectura es CISC o RISC?” dando pistas como “formato de instrucción fijo de 32 bits” o “todas las instrucciones aritméticas operan solo sobre registros”. Si ves **formato fijo + load-store** (memoria solo por LOAD/STORE), es RISC casi seguro. Si ves **formato variable + una instrucción aritmética puede leer de memoria directamente**, es CISC.

Error frecuente y con trampa real de examen: pensar que “RISC es más simple, por lo tanto siempre más lento” o que “CISC es viejo y ya no se usa”. Dato curioso (y evaluable): los procesadores Intel modernos (x86) son CISC “hacia afuera” (el programador ve el set de instrucciones x86 clásico) pero por dentro traducen cada instrucción CISC a “micro-operaciones” tipo RISC que sí se ejecutan con pipeline eficiente. Es decir: un Pentium/Core es, en el fondo, una máquina RISC que emula un set CISC.

4. Formato de instrucción y cálculo de bits

Definición: el formato de instrucción es la forma binaria en que se codifica una instrucción para guardarla en memoria. Los campos típicos son:

- **Código de operación (opcode):** identifica qué instrucción es.
- **Modo(s) de direccionamiento:** identifica cómo interpretar cada operando (ver sección 5).
- **Operandos / registros / direcciones / inmediatos:** los datos concretos que la instrucción necesita.

4.1 PROCEDIMIENTO PARA CALCULAR CUÁNTOS BITS HACEN FALTA

Para cualquier campo que deba distinguir entre \$N\$ posibilidades distintas (instrucciones, registros, modos de direccionamiento), la cantidad mínima de bits es:

$$\text{bits} = \lceil \log_2(N) \rceil$$

(el “techo”, redondeando hacia arriba si no da exacto, porque con bits enteros no se puede tener una fracción de bit).

Ejemplo 1 — opcode: si el set de instrucciones tiene **64 instrucciones posibles**,

¿cuántos bits de opcode hacen falta?

$\log_2(64) = 6$ exacto ($2^6 = 64$). **Respuesta: 6 bits.**

Verificación: con 6 bits hay $2^6=64$ combinaciones posibles → alcanzan justo para las 64 instrucciones, sin desperdicio ni faltante.

Ejemplo 2 — registros: con **16 registros** (R0 a R15), ¿cuántos bits hacen falta para identificar un registro en la instrucción?

$\log_2(16) = 4$ exacto ($2^4 = 16$). **Respuesta: 4 bits.**

Este es exactamente el caso del Ejercicio 1 del Práctico 6: CPU RISC de 16 bits con 16

registros → opcode de 4 bits + registro de 4 bits, y cada campo de registro adicional también ocupa 4 bits.

Ejemplo 3 — caso NO exacto (la trampa más común): si hay **5 modos de direccionamiento** distintos, ¿cuántos bits hacen falta para el campo de modo?

$\log_2(5) \approx 2.32 \rightarrow$ no es entero, así que se redondea **hacia arriba**: 3 bits.

Verificación: con 2 bits solo hay $2^2=4$ combinaciones — no alcanza para 5 modos. Con 3 bits hay $2^3=8$ combinaciones, que sí alcanzan (y sobran 3 combinaciones sin usar, lo cual es normal y esperable).

Error de tolerancia cero: usar $\log_2(N)$ sin redondear hacia arriba cuando N no es potencia de 2. Si te preguntan bits para 5 modos de direccionamiento y ponés “2.32 bits” o redondeás para abajo a “2 bits”, la cuenta está mal: 2 bits nunca alcanzan para distinguir 5 casos distintos. Siempre verificá con la potencia de 2: ¿ $2^{\text{bits}} \geq N$? Si no se cumple, subí un bit más.

4.2 EJEMPLO INTEGRADOR: ARMANDO UN FORMATO DE INSTRUCCIÓN COMPLETO

Supongamos una CPU con:

- 32 instrucciones posibles → opcode: $\lceil \log_2(32) \rceil = 5$ bits (verificación: $2^5=32$, exacto).
- 8 registros de uso general → cada campo de registro: $\lceil \log_2(8) \rceil = 3$ bits (verificación: $2^3=8$, exacto).
- 3 modos de direccionamiento → campo de modo: $\lceil \log_2(3) \rceil = 2$ bits (verificación: $2^2=4 \geq 3$, alcanza).

Si una instrucción de formato “3 direcciones” (opcode + 3 registros, cada uno con su modo) necesitara los tres campos de registro y de modo, el ancho total sería:

$$5 + 3 \times (2+3) = 5 + 3 \times 5 = 5 + 15 = 20 \text{ bits}$$

Truco de verificación: siempre sumá los anchos de todos los campos del formato y compáralo con el ancho total de instrucción que te da el enunciado (si te lo da). Si el enunciado dice “instrucciones de 16 bits” y tu cuenta de campos te da 20, hay un error — usualmente significa que no todas las instrucciones usan todos los campos (como pasa en el Ejercicio 2 del Práctico 6, donde JMP/JZ/JN usan un inmediato grande en vez de tres registros).

5. Modos de direccionamiento

Definición: el modo de direccionamiento establece **qué significa el operando** escrito en la instrucción y **cómo se llega al valor real** a partir de él.

Vamos a usar el mismo escenario numérico para los 5 modos, así se ve claramente la diferencia. Estado inicial de memoria y registros:

Dirección de memoria	Contenido
0x10	0x20
0x20	7

Registro	Contenido
R2	0x20

5.1 INMEDIATO

Definición: el operando es el propio valor, escrito literalmente dentro de la instrucción. No hay ningún acceso a memoria ni a registro para “buscar” el dato — ya está ahí.

Ejemplo: `MOVI #5, R1` → R1 = 5 directamente. El “5” viaja pegado al código de la instrucción.

Se usa típicamente para constantes, porque las instrucciones no se pueden modificar en tiempo de ejecución (están en la zona de código).

5.2 DIRECTO A REGISTRO

Definición: el operando está en un registro, y la instrucción contiene el **identificador del registro** (no la dirección de memoria).

Ejemplo: `ADD R2, R3, R1` → si R2=0x20 y R3=1, entonces R1 = R2 + R3 = 0x21. El operando final se obtiene leyendo directamente el registro nombrado, sin pasar por memoria.

5.3 DIRECTO A MEMORIA

Definición: la instrucción contiene, literalmente, la **dirección de memoria** donde está el operando. Se lee esa dirección y ese es el valor final.

Ejemplo: `LOAD [0x20], R1`

Paso a paso: la instrucción trae la dirección 0x20 (directa, escrita en el código). Se accede a memoria en esa dirección: $\text{mem}[0x20] = 7$. Resultado: **R1 = 7**.

5.4 INDIRECTO A REGISTRO

Definición: el operando está en memoria, pero su dirección **no** está en la instrucción — está guardada dentro de un registro. La instrucción contiene el identificador de ese registro.

Ejemplo: `LOAD (R2), R1`, con $R2 = 0x20$ (según la tabla de arriba)

Paso a paso: (1) se lee el registro R2 → obtengo la dirección 0x20. (2) Se accede a memoria en esa dirección: $\text{mem}[0x20] = 7$. Resultado: **R1 = 7**.

Notar la diferencia con el directo a memoria: acá la dirección “viaja” dentro de un registro, no dentro de la instrucción, lo que permite cambiar dinámicamente a qué posición se accede (clave para recorrer arrays con un loop reutilizando la misma instrucción).

5.5 INDIRECTO A MEMORIA

Definición: la instrucción contiene la dirección de una posición de memoria que a su vez contiene la dirección del operando real (un “puntero a puntero”). Requiere **dos** accesos a memoria.

Ejemplo: `LOAD [[0x10]], R1`

Paso a paso: (1) la instrucción trae la dirección 0x10 (directa). (2) Se accede a memoria en 0x10: $\text{mem}[0x10] = 0x20$ — esto NO es el dato, es otra dirección. (3) Se accede a memoria en esa segunda dirección: $\text{mem}[0x20] = 7$. Resultado: **R1 = 7**.

Este modo prácticamente no se implementa en arquitecturas reales (Intel, MIPS, SPARC no lo tienen) porque requiere dos accesos a memoria en serie — es lento — pero es importante saber definirlo porque aparece en las clasificaciones teóricas.

5.6 TABLA RESUMEN DE LOS 5 MODOS

Modo	¿Qué contiene la instrucción?	¿Cuántos accesos a memoria?	Resultado del ejemplo
Inmediato	El valor mismo	0	Valor literal (ej.: 5)
Directo a registro	Identificador de registro	0	Contenido del registro
Directo a memoria	Dirección de memoria	1	mem[0x20] = 7
Indirecto a registro	Identificador de registro (que contiene una dirección)	1	mem[R2] = mem[0x20] = 7
Indirecto a memoria	Dirección de una dirección	2	mem[mem[0x10]] = mem[0x20] = 7

Error frecuente: confundir “directo a memoria” con “indirecto a registro” porque ambos terminan accediendo una vez a memoria. La diferencia NO es la cantidad de accesos a memoria — es **de dónde sale la dirección**: en el directo a memoria la dirección está escrita fija en la instrucción (no cambia sin recompilar); en el indirecto a registro la dirección está en un registro, que puede cambiar en tiempo de ejecución (por eso es indispensable para recorrer arrays).

6. Ciclo de instrucción

Definición: es la secuencia de pasos que ejecuta la Unidad de Control para procesar UNA instrucción completa, desde que se busca en memoria hasta que se guarda su resultado.

6.1 LAS 5 ETAPAS

```

FETCH → DECODE → READ → EXECUTE → WRITE
(PC)   (IR)   (mem)  (ALU)   (destino)

```

Etapa	Qué hace	Qué registros/buses usa
Fetch	Busca la próxima instrucción en memoria, usando el PC (Program Counter / IP) como dirección	Bus de direcciones ← PC; bus de datos → IR; se incrementa el PC
Decode	Interpreta el opcode leído en IR: qué operación es, qué modos de direccionamiento y qué operandos tiene	Solo el registro IR , internamente en la UC
Read	Busca los operandos: si están en registros, los copia a registros auxiliares de la ALU; si están en memoria, hace un acceso de lectura	Banco de registros y/o bus de direcciones/datos hacia memoria
Execute	La ALU ejecuta la operación indicada sobre los operandos ya disponibles	ALU y sus registros auxiliares (A Latch, B Latch)
Write	Guarda el resultado en el registro destino o en memoria	Banco de registros, o bus de direcciones/datos hacia memoria

No todas las instrucciones usan las 5 etapas. Una instrucción con operandos en registros (ej.: `ADD R1, R2, R3`) no necesita **Read** desde memoria. Una instrucción que no guarda resultado en ningún lado accesible (poco común) no necesitaría **Write**. Esta observación es clave para entender por qué el pipeline (tema siguiente del curso) no siempre tiene todas las etapas ocupadas.

Ejemplo — suma con operando en memoria (direccionamiento directo): `ADD [dir], AC` (sumar el contenido de memoria en `dir` al Acumulador)

1. **Fetch:** se lee la instrucción desde la dirección que indica PC, se trae al IR, se incrementa PC.
2. **Decode:** se reconoce que es un ADD con direccionamiento directo a memoria; se extrae el campo dirección de la instrucción.
3. **Read:** se usa esa dirección para leer memoria y traer el operando al registro auxiliar de la ALU (vía MBR).
4. **Execute:** la ALU suma ese operando con el contenido actual de AC.
5. **Write:** el resultado se guarda de vuelta en AC.

Por qué importa esta descomposición en 5 etapas: es exactamente la base del **pipeline** que se estudia después en el curso — cada etapa se convierte en una “estación” de una línea de montaje, donde varias instrucciones distintas pueden estar en etapas distintas

simultáneamente. Entender bien estas 5 etapas ahora ahorra mucho esfuerzo más adelante.

6.2 EL “FETCH” NO SIEMPRE DURA LO MISMO

Error frecuente de examen: asumir que todas las etapas duran lo mismo. En arquitecturas CISC (formato variable), el Fetch puede requerir más de un acceso a memoria si la instrucción es larga, y el Execute puede tardar mucho más en una instrucción compleja que en otra simple. Esto es precisamente una de las razones por las que RISC (formato fijo, instrucciones simples) es más fácil de “empipelinar” eficientemente — todas las etapas tienden a durar lo mismo.

7. Lógica cableada vs. lógica microprogramada

Definición: son las dos filosofías para construir la Unidad de Control.

7.1 LÓGICA CABLEADA (HARDWIRED)

La Unidad de Control se diseña como cualquier circuito secuencial: se especifica un diagrama de estados que contempla **todas** las combinaciones posibles de instrucción/estado, y se traduce directamente a compuertas lógicas (flip-flops + lógica combinatoria), típicamente modelado como una máquina de Mealy.

- **Ventaja:** máxima velocidad — no hay indirección, las señales de control salen directo de la lógica combinatoria.
- **Desventaja:** poca flexibilidad. Cualquier cambio en el set de instrucciones obliga a **rediseñar el circuito completo**.

7.2 LÓGICA MICROPROGRAMADA

La Unidad de Control se construye como un autómata simple que **ejecuta**, para cada instrucción, un pequeño programa almacenado (el **microprograma**), formado por una secuencia de **microinstrucciones**. Cada microinstrucción contiene los valores concretos (0/1) de las señales de control necesarias para un paso del ciclo de instrucción.

- **Ventaja:** gran flexibilidad — para cambiar el comportamiento de una instrucción de la CPU (o agregar una nueva) alcanza con **reescribir el contenido de la ROM del microprograma**, sin tocar el hardware.
- **Desventaja:** cada microinstrucción debe leerse de una ROM interna (el “Control Store”), lo que agrega latencia comparado con la lógica cableada.

Cómo diferenciarlos rápido en un enunciado: si el enunciado habla de “microinstrucciones”, “Control Store”, “MIR (Micro Instruction Register)” o “microprograma”, es lógica microprogramada. Si habla directamente de “diagrama de estados”, “Mealy”, “tabla de transición y salidas” implementada en compuertas, es lógica cableada.

7.3 EL MICRO-ENSAMBLADOR

Como programar directamente en códigos binarios de microinstrucción es muy engorroso, se usa un **micro-ensamblador** (ej.: MAL — Micro Assembler Language, propuesto por Tanenbaum para la MIC-1) que permite escribir microinstrucciones con una notación más legible, del estilo:

```
ac := ac + d          // suma el registro D al acumulador
if n then goto 33     // salta si el bit N quedó en 1
```

Esto no cambia la naturaleza de la microprogramación: sigue siendo, en el fondo, un patrón de bits guardado en una ROM interna — el micro-ensamblador solo facilita escribirlo y entenderlo.

Error frecuente: pensar que la lógica microprogramada es “más simple” en el sentido de “circuito más chico”. Es al revés en cuanto a diseño conceptual: la ventaja de la microprogramación es la **flexibilidad de mantenimiento** (cambiar la ROM sin rediseñar hardware), no la simplicidad del circuito en sí. De hecho, requiere un subsistema adicional completo: la ROM de microprograma, el MPC (Micro Program Counter), el MIR y la lógica de secuenciamiento.

8. Ejercicios de repaso rápido (con respuesta)

Tapá la columna de respuesta y resolvé antes de mirar.

Enunciado	Respuesta
Bits de opcode para 64 instrucciones posibles	$\log_2(64)=6 \rightarrow 6 \text{ bits } (2^6=64, \text{ exacto})$
Bits para identificar 1 de 16 registros	$\log_2(16)=4 \rightarrow 4 \text{ bits}$
Bits para 5 modos de direccionamiento distintos	$\log_2(5) \approx 2.32 \rightarrow \text{se redondea hacia arriba} \rightarrow 3 \text{ bits } (2^3=8 \geq 5)$
Bits para 3 modos de direccionamiento distintos	$\log_2(3) \approx 1.58 \rightarrow 2 \text{ bits } (2^2=4 \geq 3)$
¿Qué modo de direccionamiento usa el propio valor escrito en la instrucción, sin ningún acceso a memoria?	Inmediato
¿Qué modo requiere 2 accesos a memoria?	Indirecto a memoria
¿Qué distingue al indirecto a registro del directo a memoria?	En indirecto a registro la dirección está en un registro (puede cambiar en runtime); en directo a memoria la dirección está fija en la instrucción
¿Cuál de las 5 etapas del ciclo de instrucción usa el PC?	Fetch
¿Cuál etapa no se ejecuta si los operandos ya están en registros?	Read
CISC vs RISC: ¿cuál tiene formato de instrucción de largo fijo?	RISC
CISC vs RISC: ¿cuál permite que una instrucción aritmética lea directamente un operando en memoria?	CISC
¿Qué tipo de Unidad de Control usa "Control Store" y "microinstrucciones"?	Microprogramada
Registro que contiene la próxima instrucción a ejecutarse	PC / IP (Program Counter / Instruction Pointer)
¿El PC es totalmente visible, parcialmente visible o interno?	Parcialmente visible (cambia por efecto de las instrucciones, pero no se nombra como operando explícito)

Enunciado	Respuesta
Bits necesarios para un campo que distinga 1000 casos posibles	$\log_2(1000) \approx 9.97 \rightarrow 10 \text{ bits } (2^{10} = 1024 \geq 1000)$

9. Resumen ultra-rápido (2 minutos)

- **Von Neumann:** CPU + Memoria + E/S conectados por bus (direcciones, datos, control). El programa es un dato más, guardado en memoria.
- **CPU:** ALU (ejecuta operaciones) + Unidad de Control (ejecuta el ciclo de instrucción) + Banco de Registros (memoria rápida y cercana).
- **Visibilidad de registros:** totalmente visibles (uso general, operandos explícitos) · parcialmente visibles (PC, SP, FLAGS — cambian sin ser nombrados) · internos (uso exclusivo de la UC, invisibles al programador).
- **CISC:** muchas instrucciones complejas, formato variable, memoria accesible desde aritmética, UC microprogramada. **RISC:** pocas instrucciones simples, formato fijo, arquitectura load-store, UC cableada.
- **Bits de un campo:** siempre $\lceil \log_2(N) \rceil$, redondeando hacia arriba y verificando que $2^{\text{bits}} \geq N$.
- **5 modos de direccionamiento:** inmediato (valor literal) · directo a registro (id. de registro) · directo a memoria (dirección fija en la instrucción) · indirecto a registro (dirección en un registro) · indirecto a memoria (dirección de una dirección, 2 accesos).
- **Ciclo de instrucción:** Fetch (busca con PC) → Decode (interpreta opcode) → Read (busca operandos) → Execute (ALU) → Write (guarda resultado).
- **Lógica cableada:** circuito fijo, rápido, difícil de modificar. **Lógica microprogramada:** ROM de microinstrucciones, flexible, algo más lenta.